Programme Name and Semester:-
Bachelor of Computer Applications (BCA) - Semester IV

Course Name (Course Code):-
Object Oriented Programming in JAVA (BCA40107)

Academic Session:- **2025-2026**

Study Material

(Object Oriented Programming in JAVA - BCA40107)

Table of Contents

Module 2: Class and Object Properties

1. Basic Concepts of Java Programming 2
- 1.1 Introduction to Java 2
- 1.2 Advantages of Java (Features) 2
- 1.3 Java Architecture: JDK, JRE, and JVM 3
  - Diagram: JDK, JRE, and JVM Relationship 3
- 1.4 Byte-code and the Compilation Process 4
  - Diagram: Java Execution Flow 4

2. Fundamentals of Java 5
- 2.1 Data Types 5
- 2.2 Type Casting 5
- 2.3 Access Specifiers 6
- 2.4 Operators and Control Flow 6
- 2.6 Arrays 8

3. Class and Object 9
- 3.1 Defining a Class 9
- 3.2 Creating an Object 10
  - Diagram: Class vs Object Memory Model 11
- 3.3 Constructors 11
- 3.4 Finalize and Garbage Collection 12

4. Advanced Object Concepts 13
- 4.1 Method Overloading 13
- 4.2 The this Keyword 13
- 4.3 Passing Objects as Parameters 14
- 4.4 Methods Returning Objects 14
- 4.5 Call by Value and Call by Reference 15
- 4.6 Static Variables and Methods 15

Name of the Faculty:- **Tridib Paul**

Designation and Department:- **Assistant Professor (Computational Sciences)**

Brainware University, Kolkata

4.7 Nested and Inner Classes 16
5. String Handling 17
5.1 String Class 17
5.2 Concept of Mutable and Immutable Strings 17
Diagram: String Constant Pool (SCP) 18
5.3 Command Line Arguments 18
6. Basics of I/O Operations 19
6.1 I/O Stream Hierarchy 19
Diagram: Byte Stream vs Character Stream 19
6.2 Keyboard Input using BufferedReader 20
6.3 Keyboard Input using Scanner Class 21
7. Practice Questions 22
Part A: Multiple Choice Questions (MCQs) 22
Part B: Short Answer Questions 26
Part C: Long Answer Questions 26
8. Answer Key 27
Part A: MCQ Solutions 27
Part B: Short Answer Solutions 28
Part C: Long Answer Solutions 29

1. Basic Concepts of Java Programming

1.1 Introduction to Java

Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It was developed by James Gosling at Sun Microsystems (now acquired by Oracle) in 1995. The core philosophy of Java is **WORA** (Write Once, Run Anywhere).

1.2 Advantages of Java (Features)

1. **Platform Independent:** Java code runs on any platform (Windows, Linux, Mac, Android) that has the Java Virtual Machine (JVM).
2. **Object-Oriented:** Everything in Java is treated as an object (except primitives). It supports OOP principles like Encapsulation, Inheritance, and Polymorphism.

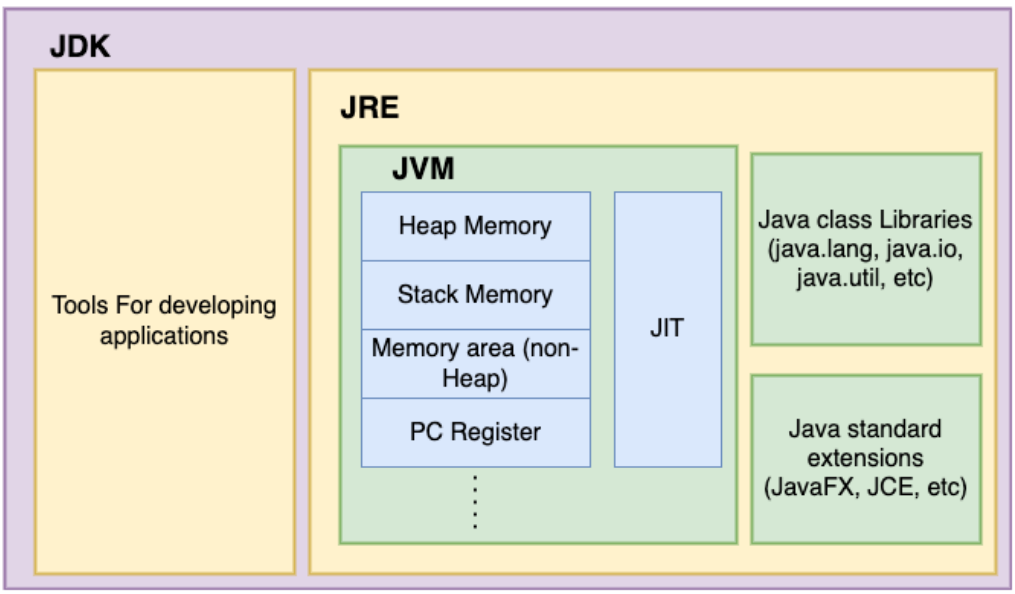
3. **Simple and Secure:** Java syntax is derived from C++ but removes complex features like pointers and explicit memory allocation, making it safer. It uses a "Sandbox" model to prevent unauthorized access to system resources.
4. **Robust:** Strong memory management (Garbage Collection) and exception handling make Java robust and crash-resistant.
5. **Multithreaded:** Allows concurrent execution of two or more threads for maximum CPU utilization.
6. **Distributed:** Supports distributed computing via RMI and EJB, allowing data to be distributed across networks.

1.3 Java Architecture: JDK, JRE, and JVM

To understand Java, one must distinguish between JDK, JRE, and JVM.

- **JDK (Java Development Kit):** The development kit required to develop Java applications. It includes JRE + Development Tools (Compiler `javac`, Debugger `jdb`, Archiver `jar`).
- **JRE (Java Runtime Environment):** The minimum requirement to run a Java application. It includes JVM + Library Classes (`java.lang`, `java.util`, etc.).
- **JVM (Java Virtual Machine):** The abstract machine that interprets the bytecode and converts it to machine-specific code.

Diagram: JDK, JRE, and JVM Relationship



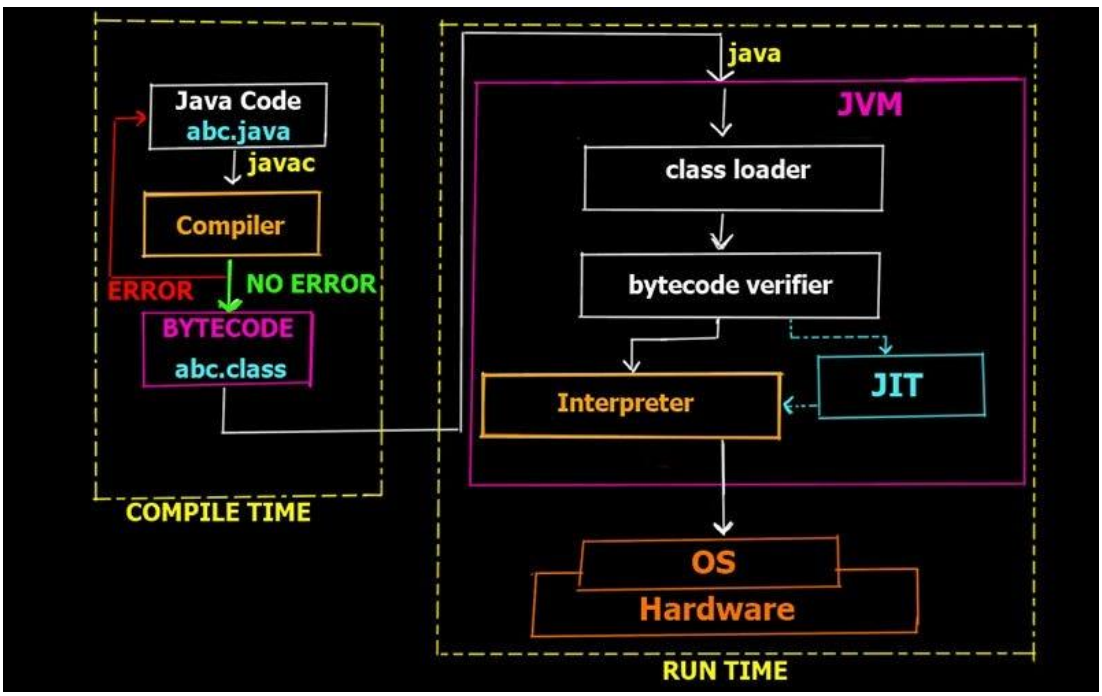
JDK	JRE	JVM
JDK stands for Java Development Kit	JRE stands for Java Runtime Environment	JVM stands for Java Virtual Machine
Java Development kit is used for developing Java applications and applets.	Java Runtime Environment is provided by Java to run the applications.	Java Virtual Machine is a specification that provides a run-time environment for Java applications and describes the requirement of JVM implementation.
It is platform-dependent.	It is platform-dependent.	It is platform-independent.
Implementation is JDK=JRE+Development tools	Implementation is JRE=JVM+libraries	Implementation is only a runtime environment to execute the code.
JDK consists of many development tools.	JRE consists of class libraries and other files.	JVM does not consist of any software tools.

1.4 Byte-code and the Compilation Process

Unlike C/C++, Java does not compile directly to machine code (0s and 1s) native to the operating system. Instead, it compiles to an intermediate format called **Bytecode**.

1. **Source Code (.java):** Human-readable code written by the programmer.
2. **Compiler (`javac`):** Converts the source code into Bytecode (.class).
3. **JVM:** The OS-specific JVM takes the bytecode and translates it line-by-line into machine code using an **Interpreter** or **JIT (Just-In-Time) Compiler**.

Diagram: Java Execution Flow



- **Why Bytecode?** Bytecode allows Java to be platform-independent. The JVM serves as a translator between the generic bytecode and the specific hardware.

2. Fundamentals of Java

2.1 Data Types

Java is strongly typed. Every variable must have a declared data type.

1. **Primitive Data Types:** These are not objects and store the actual value.

Type	Description	Size	Example
byte	Integer small	1 byte	100
short	Integer medium	2 bytes	5000
int	Integer default	4 bytes	100000
long	Integer large	8 bytes	15000000000L
float	Single precision decimal	4 bytes	10.5f
double	Double precision decimal	8 bytes	10.55555
char	Single character/Unicode	2 bytes	A'
boolean	True or False	1 bit	TRUE

2. **Reference Data Types:** These store the address (reference) of the object.
    - Classes, Arrays, Interfaces, Strings.

2.2 Type Casting

The process of converting one data type into another.

- **Widening (Implicit):** Converting a smaller type to a larger type (e.g., `int` to `double`). No data loss.

- **Narrowing (Explicit):** Converting a larger type to a smaller type (e.g., `double` to `int`). Requires manual casting and may result in data loss.

```java
int i = 100;

long l = i; // Implicit casting



double d = 10.5;

int x = (int) d; // Explicit casting, x becomes 10
```

2.3 Access Specifiers

These keywords define the scope of a class, method, or variable.

- `public`: Accessible from everywhere (other packages, classes).
- `private`: Accessible only within the class where it is declared.
- `protected`: Accessible within the same package and subclasses (even in other packages).
- `default`: (No keyword) Accessible only within the same package.

2.4 Operators and Control Flow

Operators:

- **Arithmetic:** `+`, `-`, `*`, `/`, `%` (Modulus).
- **Relational:** `==`, `!=`, `<`, `>`.
- **Logical:** `&&` (Short-circuit AND), `||` (Short-circuit OR), `!`.
- **Bitwise:** `&`, `|`, `^`, `~`, `<<`, `>>`.

Control Statements:

If-Else

```java
int age = 18;
if (age >= 18) {
   System.out.println("Eligible to vote");
} else {
   System.out.println("Not eligible");
}
```

Switch Case

```java
int day = 3;
switch(day) {
   case 1: System.out.println("Monday"); break;
   case 2: System.out.println("Tuesday"); break;
   case 3: System.out.println("Wednesday"); break;
   default: System.out.println("Other day");
}
```

Loops

- **For Loop:** Used when the number of iterations is known.

```java
for(int i = 0; i < 5; i++) {
   System.out.println(i);
}
```

- **While Loop:** Used when iterations are unknown, based on a condition.

```java
int i = 0;
while(i < 5) {
   System.out.println(i);
   i++;
}
```

- **Do-While Loop:** Executes at least once before checking the condition.

```java
int i = 0;
do {
    System.out.println(i);
    i++;
} while(i < 5);
```

- **Enhanced For-Loop:** Used to traverse arrays/collections.

```java
int[] arr = {10, 20, 30};
for(int val : arr) {
    System.out.println(val);
}
```

2.6 Arrays

An array is a container object that holds a fixed number of values of a single type. In Java, arrays are dynamically created objects.

```java
// Declaration
int[] myArray;
// Instantiation
```

```
myArray = new int[5];

// Initialization

myArray[0] = 10;



// Multidimensional Array

int[][] matrix = new int[2][3];

matrix[0][0] = 1;
```

3. Class and Object

3.1 Defining a Class

A class is a user-defined blueprint or prototype from which objects are created. It represents a set of properties or methods that are common to all objects of one type.

Structure of a Class:

1. **Fields (Variables):** State.
2. **Constructors:** Initialize objects.
3. **Methods:** Behavior.

java

```
public class Car {

   // Fields (State)

   String model;

   String color;

   int price;
```

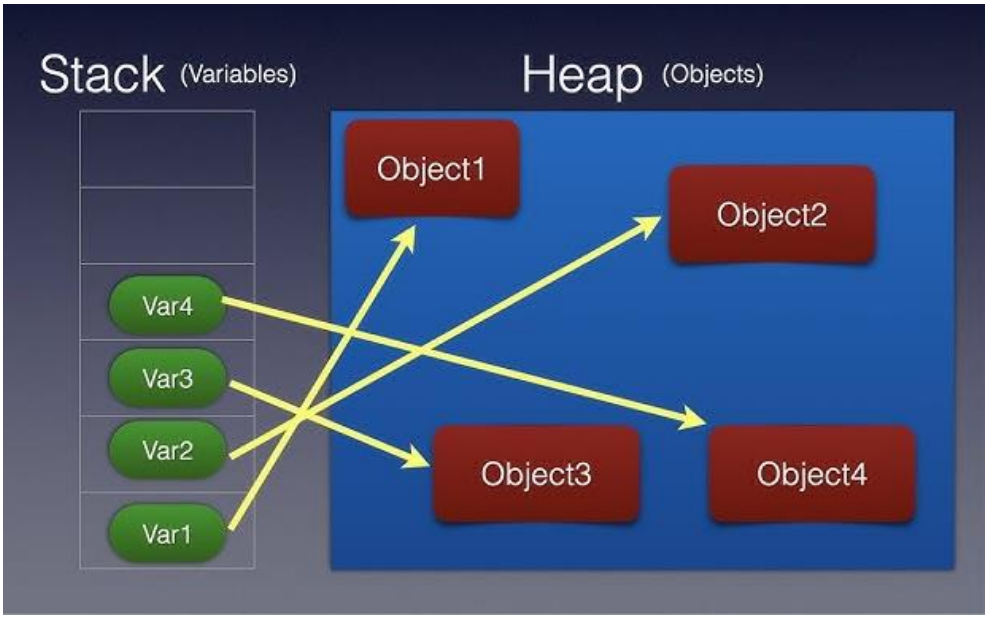
```
    // Default Constructor
    Car() {
        model = "Unknown";
        color = "White";
    }

    // Method (Behavior)
    void drive() {
        System.out.println(model + " is driving.");
    }
}
```

3.2 Creating an Object

Objects are instances of classes. The `new` keyword allocates memory in the Heap and returns a reference.

```java
Car c1 = new Car();
c1.model = "Tesla";
c1.drive(); // Output: Tesla is driving.
```

Diagram: Class vs Object Memory Model



3.3 Constructors

A constructor is a block of code that initializes a newly created object. It has the same name as the class and **no return type**.

Types:

1. **No-Argument (Default) Constructor:** If no constructor is defined, Java provides a default one.
2. **Parameterized Constructor:** Used to initialize specific fields with user-defined values.

Constructor Chaining:

Calling one constructor from another constructor using `this()`.

java

```java
class Box {

   int width, height;


   Box() {

       this(10, 10); // Calls parameterized constructor

       System.out.println("Default constructor called");
```

```
    }


    Box(int w, int h) {
        width = w;
        height = h;
        System.out.println("Parameterized constructor");
    }
}
```

3.4 Finalize and Garbage Collection

- **Garbage Collection (GC):** In languages like C++, the programmer must manually free memory (`free` or `delete`). In Java, the JVM automatically destroys objects that are no longer reachable.

- **Making an Object Eligible for GC:**

```
java

Integer i = new Integer(10);

i = null; // Object is now eligible for GC
```

- **`finalize()`** Method: Just before an object is destroyed by the Garbage Collector, the JVM calls the `finalize()` method. It is used to perform cleanup activities (closing a file, database connection).
    - Syntax: `protected void finalize() throws Throwable`

4. Advanced Object Concepts

4.1 Method Overloading

When a class has multiple methods with the same name but different parameters (number, type, or order), it is called Method Overloading. It is a form of **Compile-time Polymorphism**.

Rules:

1. Changing only the return type does NOT overload a method.
2. Parameters must differ.

```java
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) { // Different data type
        return a + b;
    }
    int add(int a, int b, int c) {   // Different count
        return a + b + c;
    }
}
```

4.2 The `this` Keyword

`this` is a reference variable that refers to the current object.

Uses:

1. **To resolve ambiguity between instance variables and local variables.**
2. **To invoke the current class constructor: `this(args)`.**
3. **To return the current class object: `return this;`.**

```java
class Student {

    int roll;

    Student(int roll) {

        this.roll = roll; // this.roll = instance, roll = local
```

```
    }

}
```

4.3 Passing Objects as Parameters

Objects can be passed to methods. Since object variables store references, passing an object means passing the reference.

```java
class Swap {

    int a, b;

    void swapValues(Swap s) {

        int temp = s.a;

        s.a = s.b;

        s.b = temp;

    }

}
```

4.4 Methods Returning Objects

Methods can also return objects of any type.

```java
class CarFactory {

    Car createCar(String name) {

        Car c = new Car();

        c.model = name;

        return c; // Returning an object reference

    }

}
```

4.5 Call by Value and Call by Reference

- **Call by Value:** Original value is NOT modified. A copy of the value is passed. (Used for primitives).
- **Call by Reference:** Original value IS modified. The reference (address) of the object is passed.

Crucial Java Concept: Java is **strictly Pass-by-Value**.
However, when passing an object, the *value of the reference* is passed.

- If you change the *state* (data) of the object using the reference, the original object changes.
- If you reassign the reference variable itself (e.g., `s = null`), the original object remains unchanged.

4.6 Static Variables and Methods

The `static` keyword is used for memory management. Static members belong to the **Class**, not the Instance.

- **Static Variable:** One copy is shared among all instances.
- **Static Method:** Can be called without creating an object (`ClassName.method()`). Cannot use `this` or access non-static members directly.
- **Static Block:** Executed only once, when the class is loaded into memory. Used for static initialization.

```java
class Test {

   static int count = 0;


   // Static Block

   static {

       System.out.println("Class Loaded");

       count = 5;

   }


   static void show() {
```

```
        System.out.println(count);

    }

}
```

4.7 Nested and Inner Classes

A class defined inside another class.

Types:

1. **Member Inner Class:** Non-static, can access private members of the outer class.

```java
class Outer {

   class Inner { ... }

}
```

2. **Static Nested Class:** Can only access static members of the outer class.

```java
class Outer {

   static class Nested { ... }

}

// Usage: Outer.Nested obj = new Outer.Nested();
```

3. **Local Inner Class:** Defined inside a method.
4. **Anonymous Inner Class:** Defined and instantiated in one expression (often used in Event Handling or Threads).

5. String Handling

5.1 String Class

In Java, strings are objects representing sequences of characters. The `java.lang.String` class is immutable (read-only).

Important String Functions:

- `int length()`: Returns string length.
- `char charAt(int index)`: Returns character at index.
- `boolean equals(String s)`: Compares content (case-sensitive).
- `boolean equalsIgnoreCase(String s)`: Ignores case.
- `String toUpperCase() / toLowerCase()`: Case conversion.
- `String trim()`: Removes leading/trailing whitespace.
- `String substring(int begin, int end)`: Extracts a portion.

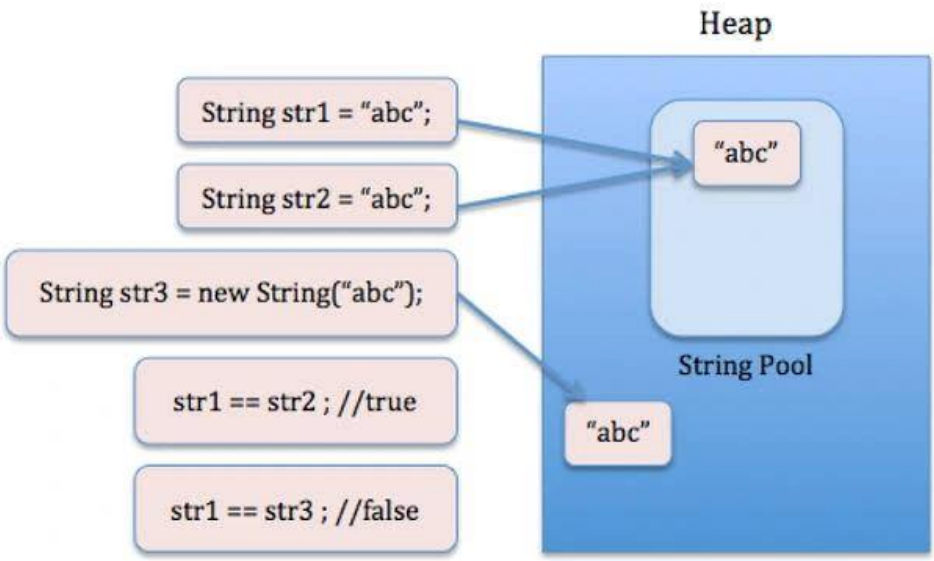
5.2 Concept of Mutable and Immutable Strings

1. **Immutable (String):** Once created, cannot be modified.
   - *Mechanism:* When you "change" a string (e.g., `s = s + "a"`), Java actually creates a NEW object in memory and discards the old one.
   - *Use Case:* When data should not change (e.g., Database URLs, keys in HashMap).
2. **Mutable (StringBuilder / StringBuffer):** Can be modified without creating new objects.
   - **StringBuffer:** Thread-safe (synchronized), slower. Used in multi-threaded environments.
   - **StringBuilder:** Not thread-safe, faster. Used in single-threaded loops.

```java
String s = "Hello";

s.concat(" World"); // String is immutable

System.out.println(s); // Prints "Hello"
```

```
StringBuilder sb = new StringBuilder("Hello");

sb.append(" World"); // String is mutable

System.out.println(sb); // Prints "Hello World"
```

Diagram: String Constant Pool (SCP)



5.3 Command Line Arguments

Arguments passed to the main method from the console during execution.

java

```
public class CmdLine {

   public static void main(String[] args) {

       // args[0] is the first argument

       System.out.println("Argument 1: " + args[0]);

   }

}
```

```
// Compile: javac CmdLine.java

// Run: java CmdLine Brainware University

// Output: Argument 1: Brainware
```

6. Basics of I/O Operations

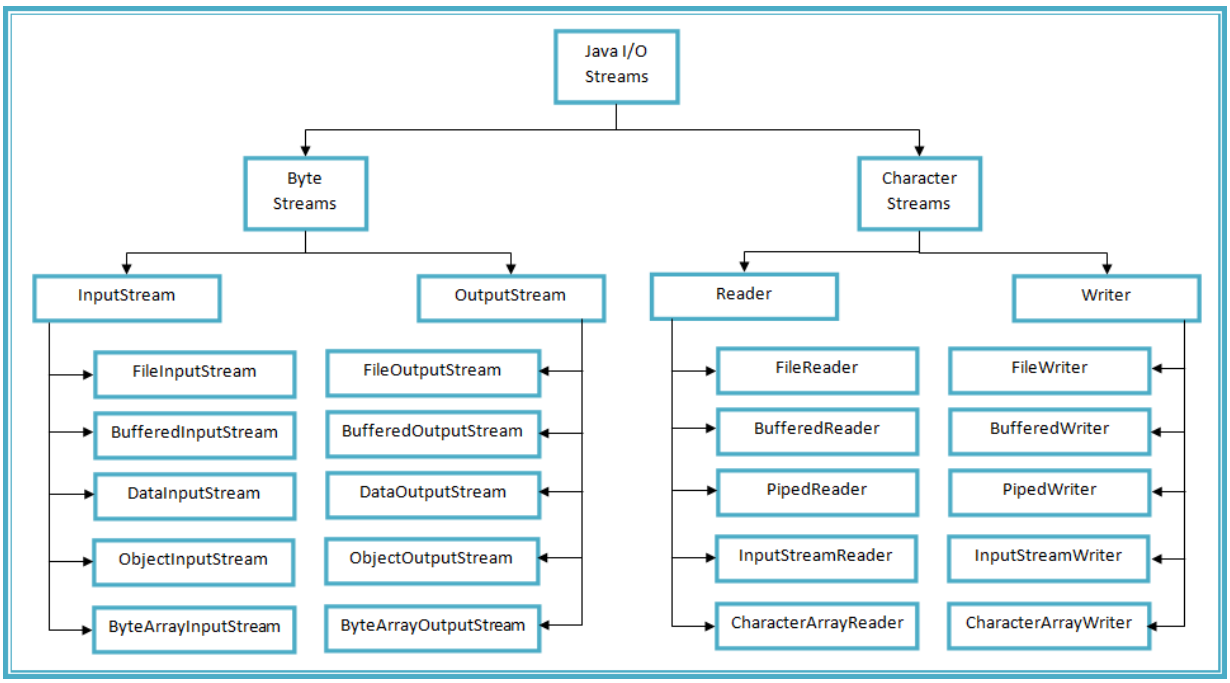
6.1 I/O Stream Hierarchy

Java uses Streams to perform I/O operations. A stream is a sequence of data.

- **Input Stream:** Reads data (Source -> Program).
- **Output Stream:** Writes data (Program -> Destination).

Diagram: Byte Stream vs Character Stream

Aspect	Character Stream	Byte Stream
Data type handled	16-bit Unicode characters (text)	8-bit raw data (binary)
Classes end with	Reader / Writer	InputStream / OutputStream
Suitable for	Text files, Unicode data	Images, audio, video, binary files
Conversion	Converts bytes to characters automatically	No conversion, works with raw bytes
Examples	FileReader, FileWriter	FileInputStream, FileOutputStream



6.2 Keyboard Input using BufferedReader

`BufferedReader` reads text from a character-input stream, buffering characters to provide efficient reading of characters, arrays, and lines. It requires `IOException` handling.

java

```java
import java.io.*;


public class ReadConsole {

   public static void main(String[] args) throws IOException {

       // Connect System.in (keyboard) to InputStreamReader

       InputStreamReader is = new InputStreamReader(System.in);

       // Connect InputStreamReader to BufferedReader

       BufferedReader br = new BufferedReader(is);


       System.out.println("Enter a string:");
```

```
        String str = br.readLine(); // Reads a line of text

        System.out.println("You entered: " + str);

    }

}
```

6.3 Keyboard Input using Scanner Class

`java.util.Scanner` is a simple text scanner which parses primitive types and strings using regular expressions. It is the most common way to read input in modern Java.

Methods:

- `next()`: Reads token (word).
- `nextLine()`: Reads entire line.
- `nextInt()`, `nextDouble()`: Read specific types.

java

```
import java.util.Scanner;


public class ScannerEx {

   public static void main(String[] args) {

       Scanner sc = new Scanner(System.in);


       System.out.print("Enter Name: ");

       String name = sc.nextLine();


       System.out.print("Enter Roll: ");

       int roll = sc.nextInt();


       System.out.println("Name: " + name + ", Roll: " + roll);
```

```
    }

}
```

7. Practice Questions

Part A: Multiple Choice Questions (MCQs)

1. Which of the following is NOT a Java feature?
   a) Object Oriented
   b) Use of pointers
   c) Portable
   d) Dynamic

2. The JDK does NOT include:
   a) JRE
   b) Compiler
   c) Database
   d) Debugger

3. What is the size of a long variable in Java?
   a) 4 bits
   b) 8 bits
   c) 4 bytes
   d) 8 bytes

4. Which access specifier is accessible only within the same class?
   a) public
   b) protected
   c) default
   d) private

5. Which operator is used to compare the values of two objects?
   a) ==

b) =
c) .equals()
d) .compare()

6. What will happen if you try to compile and run the following code?
   int[] arr = new int[5]; System.out.println(arr[10]);
   a) Compiles and runs fine
   b) Compile time error
   c) ArrayIndexOutOfBoundsException
   d) NullPointerException

7. Which method is used to execute the garbage collection?
   a) System.free()
   b) System.gc()
   c) Runtime.gc()
   d) System.delete()
8. Which of the following represents compile-time polymorphism?
   a) Method Overriding
   b) Method Overloading
   c) Inheritance
   d) Encapsulation

9. The this keyword refers to:
   a) The current method
   b) The parent class
   c) The current object
   d) The static variable

10. Can a constructor be final?
    a) Yes
    b) No
    c) Only static
    d) Only if it is private

11. Which class is used for mutable strings?
    a) String
    b) StringBuilder
    c) StringTokenizer
    d) Both a and b

12. Which package contains the Scanner class?
    a) java.io
    b) java.util
    c) java.lang
    d) java.net

13. What is the return type of the main method?
    a) int
    b) String
    c) void
    d) null

14. In Java, passing an object to a method is technically:
    a) Call by Value
    b) Call by Reference
    c) Call by Address
    d) None

15. Which block is executed before the main method?
    a) static block
    b) constructor
    c) instance block
    d) initializer

16. StringBuffer is:
    a) Mutable and thread-safe
    b) Immutable and thread-safe
    c) Mutable and not thread-safe
    d) Immutable and not thread-safe

17. Which command is used to compile a Java file named Test.java?
    a) java Test.java
    b) javac Test.java
    c) run Test.java
    d) javac Test

18. Which exception is thrown by readLine()?
    a) IOException
    b) SQLException
    c) RuntimeException

d) FileNotFoundException

19. The process of converting a primitive type to an object is called:
    a) Autoboxing
    b) Unboxing
    c) Serialization
    d) Casting

20. Which loop is best suited when the number of iterations is unknown?
    a) for
    b) while
    c) do-while
    d) for-each

21. Which of these data types has the highest precision?
    a) float
    b) double
    c) long
    d) int

22. What is the output of 10 % 3?
    a) 1
    b) 3
    c) 0.33
    d) 10

23. To access a member of a static inner class, we use:
    a) Outer.Inner
    b) Outer.static new Inner()
    c) new Outer().new Inner()
    d) Outer.Inner obj = new Outer.Inner();

24. Which method is used to find the length of a string?
    a) len()
    b) length()
    c) size()
    d) length

25. What is bytecode?
    a) Source code

b) Machine code
c) Intermediate code understood by JVM
d) Object code

Part B: Short Answer Questions

1. What is the difference between JDK, JRE, and JVM?
2. Why is Java called platform-independent?
3. Define Object and Class.
4. What is type casting? Give one example.
5. What is the use of the static keyword?
6. What is a constructor? Does it have a return type?
7. Explain the difference between == and .equals().
8. What is Garbage Collection in Java?
9. What is Method Overloading?
10. Differentiate between this and super.
11. What is a string constant pool?
12. Why is String immutable in Java?
13. What is the difference between Scanner and BufferedReader?
14. What is an Array? How do you declare an array in Java?
15. What is the default value of a boolean variable?
16. Explain Call by Value.
17. What is the use of the finalize() method?
18. Can we overload the main method?
19. What is a package in Java?
20. What are command line arguments?
21. What is the difference between String and StringBuilder?
22. What is the final keyword?
23. What is a nested class?
24. How do you exit a loop prematurely?
25. What is the wrapper class for int?

Part C: Long Answer Questions

1. Explain the architecture of Java. Discuss JDK, JRE, and JVM in detail with a diagram.
2. Write a detailed note on the differences between Primitive and Non-Primitive data types in Java.
3. Explain the "Object Oriented" concepts: Encapsulation, Inheritance, and Polymorphism with real-world examples.
4. What is a Constructor? Explain the different types of constructors with a Java program demonstrating Constructor Chaining.
5. Differentiate between Method Overloading and Method Overriding. Provide examples for each.

6. Write a Java program to demonstrate the use of the this keyword.
7. Explain the concept of Garbage Collection. How can you make an object eligible for Garbage Collection? Write a program using finalize().
8. Write a Java program to check if a number is Prime or Not.
9. Explain Call by Value and Call by Reference in the context of Java. Is Java strictly Call by Value? Justify.
10. What is a Static Block? Write a Java program that executes a static block before the main method.
11. Explain String handling in Java. Write a program to count the number of words in a given sentence.
12. Differentiate between String, StringBuffer, and StringBuilder. Which one is better to use in a loop? Why?
13. Write a Java program to demonstrate Multi-dimensional Array (Matrix Addition of two matrices).
14. Explain Nested Classes with examples. What is the difference between Member Inner Class and Static Nested Class?
15. Write a program to accept a student's name and roll number from the keyboard using the Scanner class and print a formatted output.
16. Write a program to read a line of text using BufferedReader and convert it to uppercase.
17. What is the difference between break and continue statements? Illustrate with a loop example.
18. Explain the hierarchy of Java I/O streams. Distinguish between Byte Streams and Character Streams.
19. Write a program to find the factorial of a number using recursion.
20. What are command line arguments? Write a program that accepts two numbers as arguments and prints their sum.
21. Explain the life cycle of an Object (Creation -> Use -> Destroy).
22. Explain Wrapper Classes. Why do we need them? Give an example of Autoboxing and Unboxing.
23. Write a Java program to sort an array of integers in ascending order.
24. What is the final keyword? How can you make a variable constant? Give an example.
25. Write a Java program to demonstrate the use of objects as parameters (Passing Object to Method).

8. Answer Key

Part A: MCQ Solutions

1. b) Use of pointers
2. c) Database
3. d) 8 bytes

4. d) private
5. c) .equals()
6. c) ArrayIndexOutOfBoundsException
7. b) System.gc()
8. b) Method Overloading
9. c) The current object
10. b) No
11. b) StringBuilder
12. b) java.util
13. c) void
14. a) Call by Value
15. a) static block
16. a) Mutable and thread-safe
17. b) javac Test.java
18. a) IOException
19. a) Autoboxing
20. b) while
21. b) double
22. a) 1
23. d) Outer.Inner obj = new Outer.Inner();
24. b) length()
25. c) Intermediate code understood by JVM

Part B: Short Answer Solutions

1. JDK vs JRE vs JVM: JDK is the development kit (includes compiler), JRE is the runtime environment (includes JVM + libraries), and JVM is the engine that executes bytecode.
2. Platform Independent: Because Java compiles to bytecode, which can run on any machine equipped with a Java Virtual Machine (JVM), regardless of the OS.
3. Object and Class: A Class is a blueprint or template from which objects are created. An Object is a real-world entity with state and behavior, an instance of a class.
4. Type Casting: Converting a variable from one data type to another. Example: double d = 10.5; int i = (int)d;
5. Static Keyword: It is used for memory management. Static members belong to the class rather than instances and can be accessed without creating an object.
6. Constructor: A block of code used to initialize an object. It has no return type, not even void.
7. == vs .equals(): == compares the reference (memory address) of objects, while .equals() compares the actual content of the objects.
8. Garbage Collection: It is the automatic process by which Java reclaims the memory used by objects that are no longer reachable or referenced.

9. Method Overloading: Defining multiple methods in a class with the same name but different parameter lists.
10. this vs super: this refers to the current object instance. super refers to the immediate parent class object.
11. String Constant Pool: A special area in the Java heap where string literals are stored. If a string already exists in the pool, it reuses the reference instead of creating a new object.
12. Immutable String: For security (e.g., passwords), thread safety, and performance (String Pool).
13. Scanner vs BufferedReader: Scanner is easier to use and parses tokens, but slower. BufferedReader is faster for reading large text and uses synchronous I/O.
14. Array: A container that holds a fixed number of values of a single type. Declaration: int[] arr = new int[10];
15. Boolean Default: false.
16. Call by Value: A method passes a copy of the value. Changes made to the parameter inside the method do not affect the original variable.
17. finalize(): It is called by the Garbage Collector just before an object is destroyed to perform cleanup activities.
18. Overloading main: Yes, Java allows method overloading for the main method, but JVM only calls the standard public static void main(String[] args).
19. Package: A namespace that organizes a set of related classes and interfaces (e.g., java.util, java.io).
20. Command Line Arguments: Arguments passed to the main method via the console during execution, stored in the args[] array.
21. String vs StringBuilder: String is immutable. StringBuilder is mutable and efficient for string manipulation.
22. final Keyword: It restricts modification: for variables (constants), for methods (cannot override), for classes (cannot inherit).
23. Nested Class: A class defined inside another class.
24. Exit Loop: Using the break statement.
25. Wrapper Class for int: Integer.

Part C: Long Answer Solutions

For long answers, refer to Sections 1 to 6.

End of Module 2